

TCL 官网智能服务助手技术文档

1. 项目背景与解决的问题

当前消费电子与家电市场已经从单一官网浏览、单一电商平台购买，演变为官网、电商平台、内容社区、直播间、线下门店、搜索引擎和 AI 助手并存的多入口场景。用户在购买、报修、咨询、对比和售后过程中需要不断切换页面与渠道。信息并不稀缺，真正稀缺的是用户的注意力和持续完成任务的耐心。

在这种环境下，品牌官网面临三个现实矛盾：

矛盾	现状	本项目目标
用户需求复杂化 vs 官网能力静态化	用户需要理解产品、服务、故障、政策和入口，但官网仍主要承担信息展示功能	让官网能够理解用户意图，主动引导用户完成服务任务
注意力稀缺 vs 体验链路割裂	用户需要在多个页面跳转、阅读、判断和操作，注意力被严重分散	用智能体压缩查找、判断和操作路径，减少无效跳转
品牌数字阵地建设 vs 用户入口分散	用户入口被平台、电商和 AI 搜索重新分配，品牌官网容易退化为静态资料库	把官网升级为可信、可解释、可持续服务用户的智能入口

传统官网常见短板包括：页面仍停留在“信息展示窗口”阶段；用户需要自己查找产品、理解参数、对比系列；客服常采用“问答 + 链接”模式，能回答但很少真正替用户完成任务；售后、维修、投诉、商用合作等路径分散，用户需要自行判断下一步。

本项目基于 Page Agent 构建“TCL 官网智能服务助手”。它不是单纯的 FAQ 问答机器人，而是一个可以读取当前网页结构、理解用户意图、调用受控网页工具并完成多步操作的官网智能体。它的目标是让官网从“被动展示信息”升级为“主动帮助用户完成任务”的智能服务体。

结合本项目现有代码实现，第一阶段聚焦 TCL 官网服务场景中的高频、低风险、可演示任务：

- 售后报修入口导航：帮助用户找到家电服务、个人产品服务、维修预约等入口。
- 人工客服入口分流：识别人工客服、投诉、不满等诉求，并在必要时转人工。
- 高频故障安全排查：对空调 E1、电视黑屏、洗衣机不脱水等问题给出安全优先的排查步骤。
- 官网产品与服务页面导航：帮助用户找到电视、空调、冰箱、洗衣机、智能门锁等产品页面。
- 商用、光伏、工程、合作类请求分流：将复杂商务类需求引导到更合适的官方入口或人工渠道。

项目特别强调“可信与可控”。为避免 AI 助手变成不可解释的黑箱，本项目坚持三项原则：

- 用户确认制：涉及购买、支付、登录、验证码、隐私信息、售后表单提交等关键动作时，必须由用户确认或手动完成。
- 可解释执行：助手需要说明推荐、拦截或转人工的理由，尽量明确依据来自页面信息、用户输入还是业务规则。
- 保留人工接管：投诉、安全风险、赔付、费用、账号、订单等复杂问题优先转人工，保留原始页面路径和退出权。

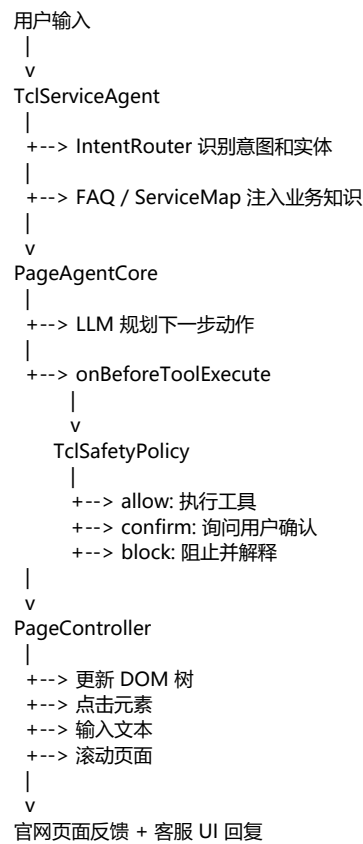
因此，本智能体不替代人工客服，不承诺维修费用、排队时间、库存、赔付结果，也不会在未经确认的情况下提交隐私信息。它解决的不是单一客服问答问题，而是数字消费场景中的系统性问题：用户需求越来越复杂、注意力越来越稀缺、官网服务路径越来越分散。最终目标是让 TCL 官网成为可执行、可解释、可信任的智能服务入口。

2. 使用的技术栈

类型	技术
编程语言	TypeScript
前端框架	React, Vite, Tailwind CSS, wouter

类型	技术
智能体框架	Page Agent monorepo 自研框架
大模型接入	OpenAI-compatible API 客户端，可配置 baseUrl、model、apiKey
DOM 工具	PageController: DOM 抽取、简化、点击、输入、滚动、元素元数据
用户界面	@page-agent/ui 客服式面板
测试框架	Vitest, happy-dom
打包方式	npm workspaces, PowerShell Compress-Archive

3. 系统架构与流程图



核心目录说明：

路径	作用
examples/tcl-official-site/src/TclServiceAgent.ts	TCL 智能服务助手组合入口
examples/tcl-official-site/src/intent/IntentRouter.ts	人工、报修、故障、导航、合作、投诉等意图路由
examples/tcl-official-site/src/safety/SafetyPolicy.ts	高风险动作确认和阻断策略
examples/tcl-official-site/src/knowledge/faq.ts	高频故障排查知识
examples/tcl-official-site/src/knowledge/serviceMap.ts	TCL 服务路径、热线和栏目知识

路径	作用
packages/core/src/PageAgentCore.ts	Agent 循环、工具执行、安全 hook
packages/page-controller/src/PageController.ts	DOM 抽取、页面动作、元素元数据
packages/ui/src/customer-service/	客服式 UI 面板
packages/website/src/pages/test-pages/	手动演示测试页面

4. 核心代码与关键逻辑说明

4.1 TclServiceAgent 组合层

TclServiceAgent 封装通用 PageAgent，注入 TCL 专属系统规则、页面规则、FAQ、服务路径、安全策略和遥测。执行时先进行意图识别，再把业务上下文写入 agent observation，最后交给底层 agent 进行多步规划和工具调用。

关键逻辑如下：

```
const routed = this.intentRouter.route(input)
this.agent.pushObservation(
  `TCL intent routing: ${routed.intent}, confidence=${routed.confidence}`
)

const matchedFaqs = findFaqByKeywords(input)
if (matchedFaqs.length > 0) {
  this.agent.pushObservation(`TCL 知识库命中 ${matchedFaqs.length} 条 FAQ`)
}

const result = await this.agent.execute(input)
```

4.2 IntentRouter 意图识别

当前 Demo 采用“规则优先”的方式完成意图分类，优点是结果可解释、稳定、便于评审复现。覆盖的主要意图包括：

- human_support：人工、客服、电话、在线客服。
- repair_service：报修、维修、售后、预约、上门。
- troubleshooting：E1、黑屏、不制冷、漏水、不脱水、异响。
- site_navigation：找、页面、产品、会员俱乐部、TCL+ APP。
- business_or_special_service：光伏、商用显示、中央空调、工程产品、加盟、商务合作。
- complaint：投诉、维权、赔偿、不满意。

4.3 SafetyPolicy 高风险动作拦截

TclSafetyPolicy 在工具真正执行前评估风险，返回 allow、confirm 或 block。

场景	处理方式
execute_javascript	阻断
登录、支付、密码、验证码	阻断
手机号、地址、订单号等敏感输入框	需要确认
提交、确认、预约类按钮	需要确认
外部客服或合作系统链接	需要确认
漏电、冒烟、进水等安全风险关键词	建议断电，并确认是否继续查找服务入口
普通导航、滚动、低风险点击	允许

这种设计让智能体的行为可解释、可追踪、可控制，避免只依赖 prompt 约束。

4.4 PageController 工具层

PageController 从真实 DOM 中抽取可交互元素，生成简化页面状态，并通过元素索引执行操作。智能体不会任意修改页面，而是通过受控工具完成任务：

```
await pageController.updateTree()
await pageController.clickElement(index)
await pageController.inputText(index, text)
await pageController.scroll(({ down: true, numPages: 1 })
```

4.5 客服式 UI

客服模式默认在首次 execute 时延迟挂载 UI。开发调试时可以使用 mode: 'developer' 禁止自动挂载 UI，便于单元测试和集成测试。

5. 运行步骤

5.1 环境要求

推荐环境：

- Node.js：建议 22.22.1 或 24 以上。
- npm：建议 11.6.3 或以上。
- 可选：OpenAI-compatible API Key，用于真实 LLM 调用。

为降低评委运行成本，项目根目录的 start.bat 已内置便携 Node.js 处理逻辑。即使 Windows 电脑没有预装 Node.js 和 npm，脚本也会自动尝试下载 Node.js 22.22.1 便携版到项目 .node\ 目录，并使用其中自带的 node.exe 和 npm.cmd 继续安装依赖、构建并启动 Demo。

如果评审环境无法访问 <https://nodejs.org>，脚本会停止并提示具体原因。此时有两种处理方式：

- 手动安装 Node.js 22.22.1 或更高版本后重新运行 start.bat。
- 在其他可联网电脑下载 Windows 版 Node.js 压缩包，解压后把包含 node.exe 和 npm.cmd 的文件夹复制到项目根目录并命名为 .node，再重新运行 start.bat。

5.2 Windows 快速启动

项目根目录提供 start.bat 快速启动脚本，适合评委直接运行：

```
.\start.bat
```

该脚本会自动完成以下步骤：

- 检查系统 Node.js 和 npm；如果缺失，会尝试下载并使用项目内便携版 Node.js。
- 执行 npm install 安装依赖。
- 启动网站开发服务器。
- 自动打开浏览器到项目首页。

启动成功后会自动打开项目首页：

```
http://localhost:5173/page-agent/
```

5.3 手动安装依赖

如果不使用 start.bat，可以手动执行：

```
npm install
```

5.4 运行单元测试

```
npx vitest run --config examples\tcl-official-site\vitest.config.ts
```

测试覆盖：

- 意图识别。
- 高风险动作安全策略。
- TCL Agent 组合层、UI 模式、遥测和生命周期。

5.5 手动启动本地 Demo

如果不使用 start.bat，可以手动启动：

```
npm start
```

浏览器访问项目首页：

```
http://localhost:5173/page-agent/
```

页面打开后，点击右下角 TCL 服务助手入口，输入示例问题：

```
我要报修电视
空调显示 E1 怎么办
我要联系人工客服
帮我找智能门锁页面
找一下商务合作页面
```

5.6 API 配置

真实 LLM 调用需要配置 OpenAI-compatible 服务。典型配置如下：

```
new TclServiceAgent({
  baseUrl: 'https://api.example.com/v1',
  model: 'your-model-name',
  apiKey: process.env.OPENAI_API_KEY,
  environment: 'production',
})
```

如果评委只验证算法逻辑和安全策略，可以直接运行 Vitest，不需要外部网络。

6. 遇到的问题及解决方案

问题	解决方案
不能把 TCL 业务规则写死进通用 core	新增 examples/tcl-official-site 业务层，通用能力仍保留在 core/page-controller/ui
只靠 prompt 约束不可靠	在 PageAgentCore 工具执行前加入 onBeforeToolExecute，由代码层返回 allow、confirm、block
报修、投诉、外链客服存在风险	TclSafetyPolicy 对提交按钮、敏感输入、外链、登录、支付、验证码统一拦截
官网页面结构可能变化	PageController 使用 DOM 抽取和元素元数据，结合文本、ARIA、placeholder、表单上下文判断
评委电脑可能没有预装 Node.js/npm	start.bat 优先使用系统 Node.js/npm，缺失时下载便携 Node.js 到 .node；如果网络无法访问 nodejs.org，则提示手动安装或复制便携 Node.js 文件夹
复杂问题不适合 AI 自动处理	投诉、安全风险、隐私信息、登录、支付等场景转人工或要求用户手动确认

7. 可演示能力清单

- 多意图识别：人工客服、报修售后、故障排查、站内导航、商用合作、投诉。
- 多步推理：先识别意图，再注入知识，再规划页面动作。
- 工具调用：点击、输入、滚动、读取页面状态。

- 安全确认：敏感字段、提交按钮、外部链接、验证码、登录、支付阻断。
- 知识增强：FAQ 和服务路径进入上下文。
- 运行验证：单元测试和本地网站测试页面。

8. 提交包说明

提交包命名格式：

赛道一_TCL官网智能服务助手_队长姓名.zip

包内包含：

- package.json、package-lock.json、TypeScript 配置和快速启动脚本。
- packages/ 下核心源码。
- examples/tcl-official-site/ Demo 源码和测试。
- docs/tcl-service-agent/ 原始设计文档。
- submissions/track-one-tcl-service-agent/ 参赛文档、示例输入输出、PDF。

重新打包命令：

```
.\scripts\build-track-one-submission.ps1 -CaptainName "队长姓名"
```